VYP Compiler Project

Authors

This project was developped by :

- Nuri Eyobed Awel. (50%)
- Julien Archambault. (50%)

Tasks distribution

Front was divided the following way:

- Lexer -> Julien.
- Parser -> Eyobed.

We developed back-end together as it is harder.

Table of Contents

- Overview
- Features
- Prerequisites
- Project Structure
- Front End
- Back End
- Building the Compiler
- Usage
- Testing
- Compilation Phases
- Examples

Overview

The VYP compiler is a multi-phase compiler that processes VYP language source files (`.vyp`) and generates executable VYPcode (`.vc`) files. The compiler performs lexical analysis, parsing, semantic analysis, and code generation.

Features

- **Object-Oriented Programming**: Support for classes and inheritance (Not functional yet)
- **Functions**: First-class functions with parameters and return values
- **Data Types**: `int` , `string` , and custom class types
- **Control Flow**: `if-else` statements and `while` loops
- **Operators**: Arithmetic (`+` , `-` , `*` , `/`), logical (`&&` , `||` , `!`), and comparison operators
- **Built-in Functions**: `print()` for output
- **Type Casting**: Cast expressions for type conversion
- **Variable Initialization**: Declaration with initialization (Extension)
- **Semantic Analysis**: Two-pass semantic checking for definitions and logic
- **Code Generation**: Generates VYPcode bytecode

Prerequisites

- **Java**: JDK 8 or higher
- **ANTLR4**: Version 4.13.2 (included in classpath)
- **Make**: For building using the Makefile
- **Bash**: For running test scripts (Linux/Unix/WSL)

Project Structure

```
compiler/
├── src/
│   ├── Compiler.java              # Main compiler driver
│   ├── parser/                    # Generated ANTLR parser/lexer
│   │   ├── VYPlanguageParser.java
│   │   ├── VYPlanguageLexer.java
│   │   └── VYPlanguageBaseVisitor.java
│   ├── semantic/                  # Semantic analysis
│   │   ├── DefinitionVisitor.java # First pass: collect definitions
│   │   ├── LogicVisitor.java      # Second pass: check logic
│   │   ├── Scope.java             # Scope management
│   │   └── Symbol.java            # Symbol table entries
│   └── codegen/
│       └── CodeGenerator.java     # VYPcode generation
├── grammar/
│   └── VYPlanguage.g4            # ANTLR grammar definition
├── tests/                         # Test programs
├── test_output/                   # Compiled test outputs
├── bin/                           # Compiled Java classes
├── Makefile                       # Build system
├── vypcomp                        # Compiler wrapper script
└── run_tests.sh                   # Test suite runner
```

Front-End Implementation

The compiler front-end performs lexical and syntactic analysis using ANTLR4.

Grammar (VYPlanguage.g4)

The grammar is defined in ANTLR4 format and consists of:

Parser Rules:

- `program` : Top-level rule accepting definitions followed by EOF
- `definition` : Classes and functions at global scope
- `functionDefinition` : Function declarations with return type, parameters, and body
- `classDefinition` : Class declarations with optional inheritance (`:` syntax)
- `statement` : Control flow (if-else, while), assignments, returns, expressions
- `expression` : Full expression grammar with precedence and associativity
- `lvalue` : Left-hand side of assignments (identifiers and field access)

Lexer Rules:

- Keywords: `class` , `if` , `else` , `while` , `return` , `int` , `string` , `void` , `new` , `this` , `super`
- Operators: Arithmetic (`+` , `-` , `*` , `/`), logical (`&&` , `||` , `!`), relational (`<` , `>` , `<=` , `>=` , `==` , `!=`)
- Identifiers: `[a-zA-Z_][a-zA-Z0-9_]*`
- Literals: Integer literals (no leading zeros except `0`), string literals with escape sequences
- Comments: Line comments (`//`) and block comments (`/* */`)
- Whitespace: Automatically skipped

Expression Precedence (lowest to highest):

1. Logical OR (`||`)
2. Logical AND (`&&`)
3. Equality (`==` , `!=`)
4. Relational (`<` , `>` , `<=` , `>=`)
5. Additive (`+` , `-`)
6. Multiplicative (`*` , `/`)
7. Unary (`!` , type cast, `new`)
8. Primary (identifiers, literals, `this` , `super` , parenthesized expressions)
9. Member access (`.` for field access and method calls)

Tools and Methods

ANTLR4 (ANother Tool for Language Recognition):

- Version 4.13.2
- Generates recursive descent parser from grammar
- Produces Java classes: `VYPlanguageLexer` , `VYPlanguageParser` , `VYPlanguageBaseVisitor`
- Uses visitor pattern for tree traversal

Visitor Pattern:

- `VYPlanguageBaseVisitor<T>` : Base class for AST traversal
- Implemented by semantic analyzers and code generator
- Provides type-safe tree walking with context-specific methods
- Each grammar rule generates a corresponding visit method

Parser Configuration:

- No listener generation (`-no-listener`)
- Visitor generation enabled (`-visitor`)
- Package organization: `parser` package for generated code
- Two-pass compilation: Parse tree generated once, visited multiple times

Back-End Implementation

The compiler back-end performs semantic analysis and generates stack-based bytecode (VYPcode).

Design Architecture

Two-Pass Semantic Analysis:

1. **Definition Pass** (`DefinitionVisitor`):
    - Collects all global function and class definitions
    - Populates global symbol table
    - Registers built-in functions (`print` , `readInt` , `readString` , `length` , `substr` , `toString`)
    - Validates main function existence (mandatory)
2. **Logic Pass** (`LogicVisitor`):
    - Type checking for expressions and statements
    - Variable usage validation (undefined references)
    - Scope management with nested scoping
    - Parameter and local variable registration
    - L-value validation for assignments

Symbol Table

Data Structures:

- `Symbol` : Represents identifiers (variables, functions, classes)
    - Fields: `name` (String), `type` (String - "int", "string", "void", or class name)
- `Scope` : Hierarchical scope manager
    - Fields: `parent` (Scope), `symbols` (HashMap<String, Symbol>)
    - Methods:
        - `define(Symbol)` : Adds symbol, detects redefinitions (exit 14)
        - `resolve(String)` : Searches current scope, then parent recursively

Scope Hierarchy:

- Global scope: Contains functions, classes, and built-ins
- Function scope: Parameters and local variables
- Block scope: Variables declared in `{ }` blocks
- Nested scoping: Inner scopes can shadow outer variables

Intermediate Representation

VYPcode (Stack-Based Bytecode):

The compiler generates VYPcode instructions for a virtual stack machine:

Registers:

- `$0` : Primary accumulator (expression results)
- `$1` : Secondary register (binary operations)
- `[n]` : Stack slots for local variables

Instruction Set:

- `SET dest, src` : Assignment/load instruction
- `ADDI` , `SUBI` , `MULI` , `DIVI` : Integer arithmetic
- `LTI` , `GTI` , `EQI` : Comparisons
- `NOT` : Logical negation
- `WRITEI` , `WRITES` : Output instructions
- `JUMPZ label, cond` : Conditional jump (jump if zero)
- `LABEL name` : Label definition for control flow

Code Generation Techniques

Stack Management:

- Dynamic stack index tracking (`stackIndex`)
- Variable offset mapping (`varOffsets` : HashMap<String, Integer>)
- Scope stack for variable shadowing (`scopeStack`)
- Temporary slots for expression evaluation

Expression Evaluation:

- Left-to-right evaluation with temporary storage
- Binary operations: Evaluate left operand → save to temp → evaluate right → operate
- Result always in `$0` register

Control Flow:

- Label generation with counter (`labelCounter`)
- If-else: Conditional jump to else/end labels
- While loops: Jump to condition check, conditional exit, unconditional back-jump
- Unconditional jumps: `JUMPZ label, 0` (always false condition)

Function Handling:

- Each function gets a label with its name
- Variable offsets reset per function
- Implicit return with `SET $0, 0` at function end

Variable Initialization Extension:

- Supports declaration with initialization: `int x = 5;`
- Generates immediate load after variable allocation
- Preserves result in `$0` for initialized expressions

Special Techniques

Operator Implementation:

- `<=` implemented as `NOT(>)` to minimize instruction set
- `>=` implemented as `NOT(<)`
- `!=` implemented as `NOT(==)`

Scope Preservation:

- Push/pop scope mechanism maintains variable visibility
- Shadowing: Inner variables hide outer ones with same name
- Scope restoration after block exit

Optimizations

Current Optimizations:

- Direct register-to-stack operations (no intermediate moves)
- Reuse of stack slots within non-overlapping scopes
- Single-pass code generation after semantic analysis
- Minimal temporary variable usage

No Optimizations (Areas for Future Enhancement):

- Constant folding not implemented
- Dead code elimination absent
- Register allocation is naive (always uses $0/$1)
- No peephole optimization
- No common subexpression elimination

Algorithms

Expression Code Generation:

```
1. For binary operations:
   - Visit left operand (result in $0)
   - Save $0 to temporary stack slot
   - Visit right operand (result in $0)
   - Load left operand from temp to $1
   - Execute operation: op $0, $1, $0

2. For comparisons:
   - Similar to binary ops
   - Special handling for <=, >=, != using NOT
```

Control Flow Generation:

```
If-Else:
1. Evaluate condition → $0
2. JUMPZ else_label, $0
3. Generate true block
4. JUMPZ end_label, 0  (unconditional)
5. LABEL else_label
6. Generate else block (if exists)
7. LABEL end_label

While:
1. LABEL loop_start
2. Evaluate condition → $0
3. JUMPZ loop_end, $0
4. Generate loop body
5. JUMPZ loop_start, 0  (unconditional)
6. LABEL loop_end
```

Scope Management:

```
1. On block entry: Push current varOffsets to scopeStack
2. Execute block: New variables added to varOffsets
3. On block exit: Pop scopeStack to restore outer varOffsets
```

Building the Compiler

Using Make

```
make
```

This will:

1. Clean up old files
2. Generate parser and lexer from the ANTLR grammar
3. Compile all Java source files

Manual Build

```
# Generate parser
cd grammar
java -jar /path/to/antlr-4.13.2-complete.jar -o ../src/parser -package parser -no-listener -visitor VYPlanguage.g4

# Compile
cd ..
javac -cp .:./bin:/path/to/antlr-4.13.2-complete.jar -d bin src/parser/*.java src/semantic/*.java src/codegen/*.java src/*.java
```

Clean Build

```
make clean
```

Usage

Compiling a VYP Program

```
./vypcomp <input_file.vyp> [output_file.vc]
```

Examples:

```
# Compile to default output (out.vc)
./vypcomp tests/hello.vyp

# Compile with custom output name
./vypcomp tests/expressions.vyp test_output/expressions.vc
```

Running the compiled code on Merlin

```
java -jar /pub/courses/vyp/vypint/vypint-1.0.jar out.vc
```

Testing

Run All Tests

```
./run_tests.sh
```

The test suite includes:

- **Success Tests**: Valid programs that should compile and run
- **Error Tests**: Programs with semantic errors that should fail compilation

Test Files

- `hello.vyp` - Basic hello world
- `expressions.vyp` - Expression evaluation and precedence
- `control.vyp` - Control flow structures
- `loops.vyp` - While loops
- `math.vyp` - Arithmetic operations
- `nested_scopes.vyp` - Variable scoping
- `error_*.vyp` - Error test cases

Compilation Phases

1. Lexical Analysis

The lexer (VYPlanguageLexer) tokenizes the input source code.

2. Syntax Analysis

The parser (VYPlanguageParser) builds an Abstract Syntax Tree (AST) according to the grammar rules.

3. Semantic Analysis

Pass 1 - Definition Visitor:

- Collects all function and class definitions
- Builds symbol tables and scopes
- Checks for redefinitions

Pass 2 - Logic Visitor:

- Type checking
- Verifies variable usage
- Validates function calls
- Checks for undefined symbols

Main Function Requirement:

- Every program must have a `main` function (exit code 14 if missing)

4. Code Generation

The CodeGenerator translates the AST into VYPcode instructions:

- Stack-based instruction set
- Label-based control flow
- Function calls and returns
- Memory management for variables

Examples

Hello World

```
void main(void) {
    print("Hello World\n");
}
```

Factorial

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

void main(void) {
    int result = factorial(5);
    print(result);
    print("\n");
}
```

Class Example

```
class Counter {
    int value;

    void increment(void) {
        this.value = this.value + 1;
    }

    int getValue(void) {
        return this.value;
    }
}

void main(void) {
    Counter c = new Counter;
    c.value = 0;
    c.increment();
    c.increment();
    print(c.getValue());
    print("\n");
}
```

License

This project is part of the VYP (Compiler Construction) course.